Convolutional Neural Networks for Imagenette Classification

Author
Nedelcu Radu - Ioan

Abstract

1 Introduction

This paper presents my solution to the first homework assignment for the Neural Networks Course, titled "The one with CNNs". It focuses on training a convolutional neural network (CNN) from scratch for image classification and using transfer learning with a pre-trained ResNet-18 model on the Imagenette dataset.

2 Dataset and Preprocessing

2.1 Imagenette

> Imagenette is a subset of 10 easily classified classes from Imagenet (tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, parachute).
>
> [Howard, 2019]

I will use Imagenette to train and evaluate convolutional neural networks on a 10-class image classification task. I will first develop a custom CNN from scratch and then use a pre-trained ResNet-18, comparing different configurations.

2.2 Processing

All experiments were conducted locally on a machine equipped with an NVIDIA GeForce RTX 5070 Ti GPU, an AMD Ryzen 7 9700X processor, and 64 GB of RAM. All models were trained and evaluated using local hardware acceleration, without relying on cloud-based computing resources. The reported training times correspond to this hardware configuration, ensuring consistency when comparing the computational performance of the tested architectures.

3 Task 2: CNN Trained from Scratch

3.1 Architecture

The initial stage of the project focused on experimenting with different Convolutional Neural Network (CNN) architectures to identify the most suitable model for the classification task. All models were trained on 160×160 images, while varying the number of convolutional layers and the number of output channels in each layer.

Eight different architectures were evaluated, ranging from relatively small networks with 3 convolutional layers to deeper networks with 4 convolutional layers and larger channel dimensions. The models were compared based on their number of parameters, best validation accuracy, best epoch, and total training time.

Table 1: Comparison of CNN architectures

ID	Model	Conv.	Channels	Parameters	Best Val.	Epoch	Time (s)
7	CNN 8	4	[64, 128, 256, 512]	1,584,458	79.03%	20	125.9
5	CNN 6	4	[32, 64, 128, 256]	405,514	75.97%	19	62.2
6	CNN 7	4	[32, 64, 128, 128]	249,738	73.48%	19	61.4
3	CNN 4	3	[64, 128, 256]	387,914	70.90%	19	106.1
4	CNN 5	4	[16, 32, 64, 128]	106,346	70.90%	20	45.1
2	CNN 3	3	[32, 64, 256]	184,202	70.32%	20	65.9
1	CNN 2	3	[32, 64, 128]	102,154	67.36%	20	58.6
0	CNN 1	3	[16, 32, 64]	28,394	62.52%	16	58.0

As shown in The architecture comparison, CNN 8 achieved the best validation accuracy of 79.03%, making it the best-performing architecture among the tested models. It also had the largest number of parameters (1,584,458) and the longest training time (125.9 seconds).

The results suggest that increasing the network's capacity generally improved classification performance. However, this came at the cost of additional computational resources and longer training times. Based on these experiments, CNN 8 was selected as the starting point for further optimization and experimentation.

3.2 Experimental configurations

Experiment	BatchNorm	Dropout	Augmentation	Best Val. Acc.	Final Val. Acc.
1	No	No	No	78.09% (20)	78.09%
2	Yes	No	No	78.22% (19)	68.13%
3	No	Yes	No	76.69% (20)	76.69%
4	No	No	Yes	**78.57% (19)**	**77.78%**
5	Yes	Yes	No	73.48% (10)	62.62%
6	Yes	No	Yes	75.24% (9)	60.87%
7	No	Yes	Yes	75.24% (20)	75.24%
8	Yes	Yes	Yes	71.39% (8)	62.14%

Table 2: Results for the eight combinations of BatchNorm, Dropout, and data augmentation using the selected CNN architecture. Best validation accuracy and corresponding epoch are shown alongside final validation accuracy.

Overall, the eight configurations achieved best validation accuracies ranging from 71.39% to 78.57%. The best result was obtained in Experiment 4, which used data augmentation without Batch Normalization or Dropout, reaching 78.57% validation accuracy at epoch 19. Batch Normalization alone achieved a similar result of 78.22%, while Dropout alone resulted in a lower accuracy of 76.69%. Combining multiple techniques did not lead to better performance; in particular, the configuration using all three methods achieved only 71.39%.

Several configurations containing Batch Normalization showed considerable fluctuations in validation accuracy, with some experiencing substantial drops between their best and final epochs. For example, Experiment 6 reached 75.24% at epoch 9 but finished at 60.87%, suggesting possible overfitting. We conclude that configurations using Batch Normalization converge faster to higher training accuracy. However, this does not necessarily translate into better validation performance.

These results indicate that the effectiveness of regularization and data augmentation depends on their combination, with data augmentation alone providing the best overall result in these experiments.

3.3 Results

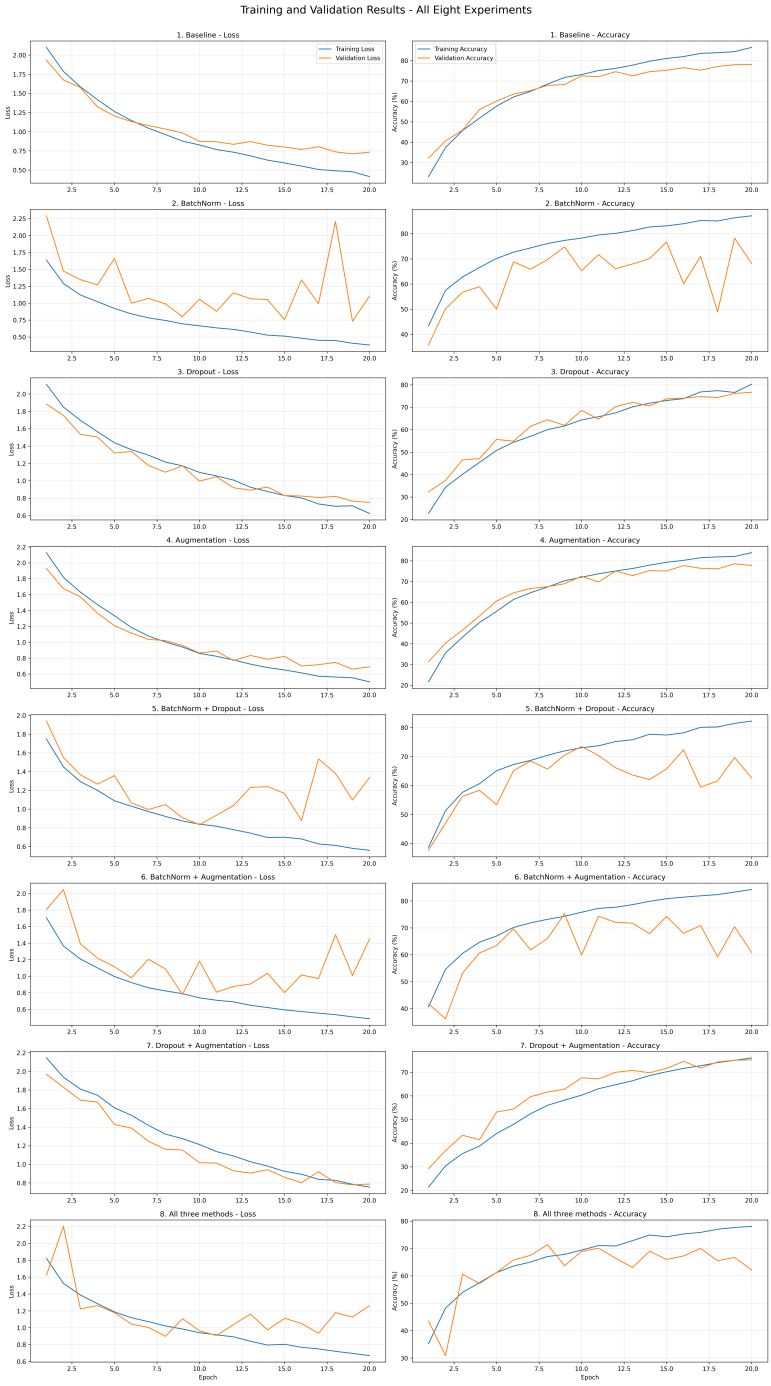


Figure 1: Training and validation loss and accuracy curves for the eight Task 2 experiments.

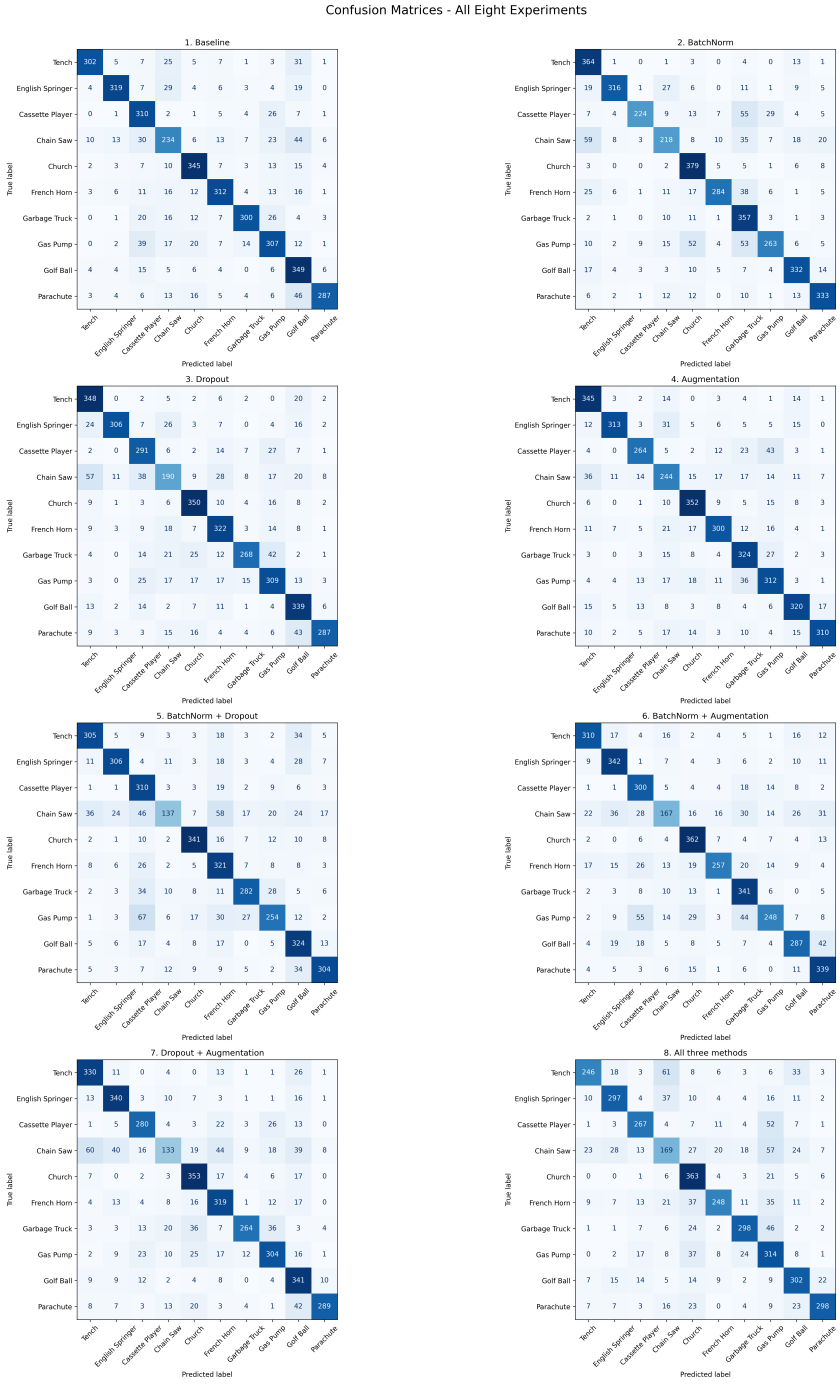


Figure 2: Confusion matrices for the eight Task 2 experiments.

The results show that all eight configurations learned to classify the Imagenette images, as indicated by the decreasing training and validation losses. Experiment 4, which used data augmentation without Batch Normalization or Dropout, achieved the highest validation accuracy of 78.57%. The confusion matrices show that most predictions fall along the main diagonal, although some classes were more frequently confused than others. In particular, Chain Saw and Gas Pump show noticeable misclassifications across several configurations. Batch Normalization did not consistently improve performance and was associated with

larger fluctuations in validation accuracy in some experiments.

Overall, data augmentation alone achieved the best result, while enabling all three techniques simultaneously did not provide an additional improvement.

4 Task 3: Transfer Learning with ResNet-18

4.1 Pretrained ResNet-18 feature extractor

ResNet (Residual Network) is a convolutional neural network architecture introduced by He et al. (2016). It uses residual connections, alsocalled skip connections, which allow information to bypass certain layers. This makes it easier to train deeper neural networks.

In their original paper, the authors describe their approach as follows:

> "We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously." [He et al., 2016]

For this assignment, I use ResNet-18, an 18-layer variant of the ResNet architecture. The model is provided by Torchvision with pre-trained weights obtained from training on ImageNet-1K, a dataset containing 1,000 image categories [PyTorch Contributors, 2026].

Instead of training the entire network from scratch, I use the pre-trained ResNet-18 as a feature extractor. Its original classification layer is replaced with a new fully connected layer containing 10 output neurons, corresponding to the Imagenette classes.

4.2 Frozen versus adapted BatchNorm statistics

According to the PyTorch documentation, Batch Normalization maintains running estimates of the mean and variance during training, which are subsequently used during evaluation

[PyTorch Contributors].

When using a pretrained ResNet-18, these statistics have already been calculated during training on ImageNet. Freezing Batch Normalization means preserving the original running statistics when adapting the model to Imagenette. In contrast, adapting Batch Normalization allows these statistics to be updated using batches from the new dataset.

The purpose of this experiment is to investigate whether adapting the normalization statistics to Imagenette improves the classification performance compared to retaining the original ImageNet statistics.

4.3 Training setup

4.4 Task 3 results

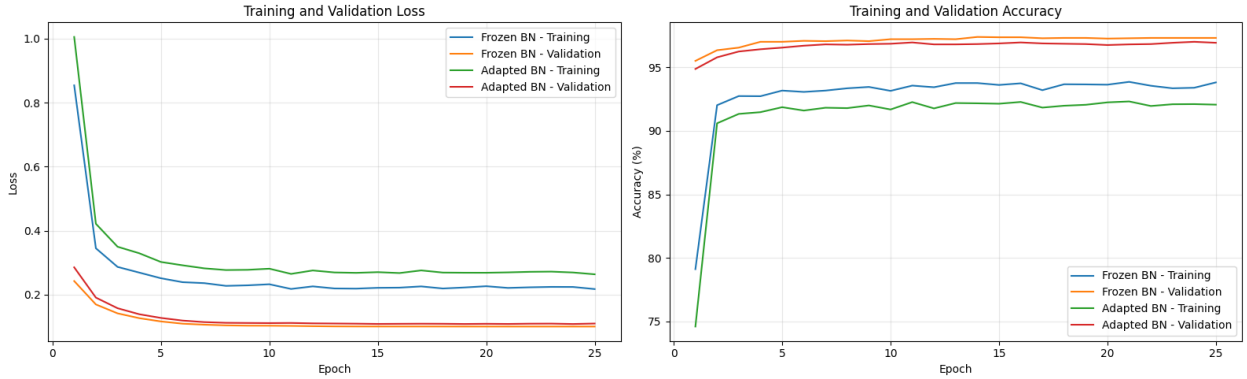


Figure 3: Validation loss and accuracy for ResNet-18 with frozen and adapted BatchNorm.

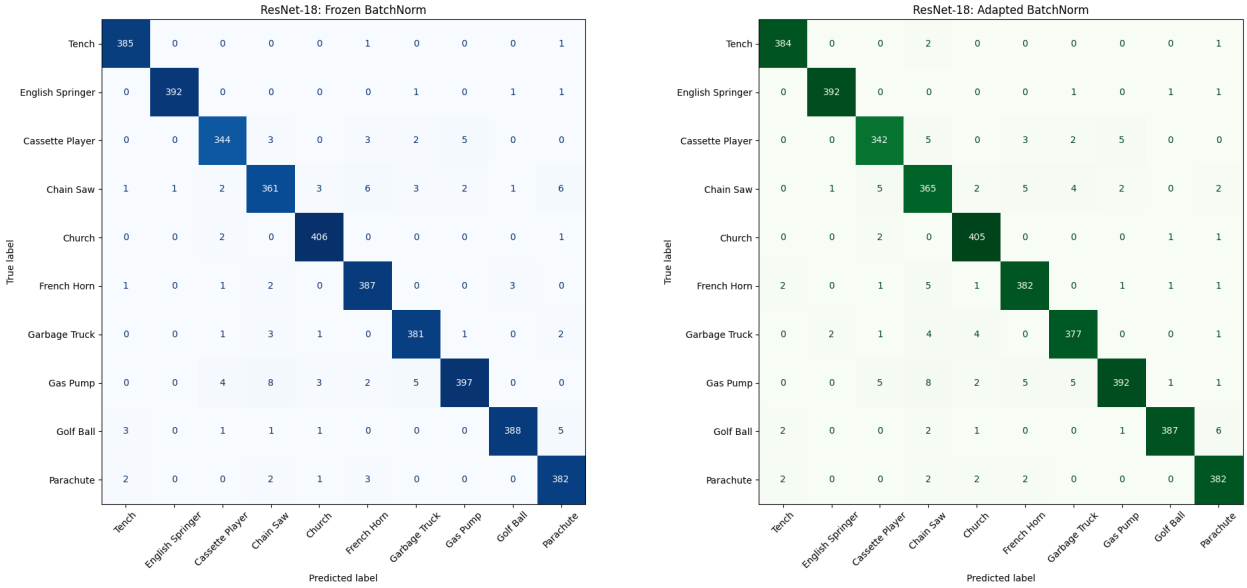


Figure 4: Confusion matrices for ResNet-18 with frozen and adapted BatchNorm.

Model	Best validation accuracy
Custom CNN (Task 2, augmentation only)	78.57%
ResNet-18, frozen BatchNorm	97.40%
ResNet-18, adapted BatchNorm	97.02%

Table 3: Best validation accuracy of the custom CNN and ResNet-18 configurations.

4.5 Conclusion

ResNet-18 outperformed the custom CNN developed in Task 2. The best custom CNN configuration, which used data augmentation without Batch Normalization or Dropout, achieved 78.57% validation accuracy. ResNet-18 with frozen Batch Normalization achieved 97.40%, an improvement of 18.83 percentage points, while the adapted Batch Normalization configuration achieved 97.02%. These results suggest that the pretrained ResNet-18 features provide a stronger starting point for Imagenette classification than the features learned by the custom CNN. Freezing the Batch Normalization statistics produced a slightly better result than adapting them, although the difference was small. Overall, ResNet-18 with frozen Batch Normalization was the best-performing configuration.

References

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

Jeremy Howard. Imagenette: A smaller subset of 10 easily classified classes from imagenet, 2019. URL https://github.com/fastai/imagenette.

PyTorch Contributors. Batchnorm2d. URL https://docs.pytorch.org/docs/stable/generated/torch.nn.modules.batchnorm.BatchNorm2d.html. Accessed November 2026.

PyTorch Contributors. Resnet-18. Torchvision Documentation, 2026. URL https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html.